

TUPLE :- ORDERED MUTABLE , ALLLOWs DUPLICATE VALUES

TUPLE IS IMMUTABLE

TUPLE IS DEFINED BY USING () PARENTHESIS

TUPLE IS ORDERED THAT MEANS IT MAINTAINS THE ORDER OF INSERTION

```
#TUPLE DECLARATION
my_tuple = (1, 2, 3, 4, 5)

#proper tuple declaration
mytuple = ("EXAMPLE",)
print(type(mytuple))

#This will be cpnsidered as a string
mytuple1 = ("example")
print(type(mytuple1))

<class 'tuple'>
<class 'str'>

#INDEXING IN TUPLES
mytuple = ("apple", "banana", "cherry")
print(mytuple[1])

banana

#LOOPING IN TUPLES

mytuple = ("apple", "banana", "cherry")

for items in mytuple:
    print(items)

apple
banana
cherry

#CONDITIONS IN TUPLES

mytuple = ("apple", "banana", "cherry")

if "banana" in mytuple:
    print("Yes, 'banana' is in the tuple")

Yes, 'banana' is in the tuple
```

#TUPLE FUNCTIONS

#1. COUNT() - This function returns the number of times a specified value occurs in a tuple.

```
mytuple = (1, 2, 3, 2, 2, 4, 5)
x = mytuple.count(2)
print(x)
```

#2. INDEX() - This function searches the tuple for a specified value and returns the position of where it was found.

```
mytuple = (1, 2, 3, 4, 5)
x = mytuple.index(3)
print(x)
```

#COMPARSION BETWEEN LISTS AND TUPLES

```
import sys
```

```
my_list = [1, 2, 3, 4, 5]
my_tuple = (1, 2, 3, 4, 5)
print("List size:", sys.getsizeof(my_list), "bytes")
print("Tuple size:", sys.getsizeof(my_tuple), "bytes")
#Lists consume more memory than tuples
```

```
import timeit
```

```
list_time = timeit.timeit(stmt="[1, 2, 3, 4, 5]", number=1000000)
tuple_time = timeit.timeit(stmt="(1, 2, 3, 4, 5)", number=1000000)
print("List creation time:", list_time)
print("Tuple creation time:", tuple_time)
#Tuples are faster than lists for creation and access operations
```